BEL C22 · JAVA TRACK

Java Revision Study Guide

Condensed notes, code snippets, mental models & memory tricks for the full course — one doc instead of 60+ files.

Class 1 — Java Fundamentals

Syntax and building blocks: the vocabulary you need before OOP makes sense.

Intro to Java

- High-level, OO, "write once run anywhere."
- Pipeline: `.java` →*javac*→ `.class` bytecode →*JVM*→ machine code.
- JDK ⊃ JRE ⊃ JVM — develop ⊃ run ⊃ execute.

Data Types

- 8 primitives (byte/short/int/long/float/double/char/boolean) — hold values directly, never null.
- Reference types (String, arrays, objects) hold an address, default `null`.
- **Trick:** primitive = box with the value; reference = label pointing at the box.

Variables & Scope

- **Local** (in a method, no default, must init), **instance** (per object, defaults), **static** (shared, one copy).
- Memory: local→stack, instance→heap, static→method area.

Type Casting

- **Widening** (small→big): automatic, no data loss. `byte→short→int→long→float→double`
- **Narrowing** (big→small): explicit `(int) d`, can lose data.
- **Trick:** "widen for free, narrow on purpose."

Operators

- Arithmetic `+ - * / %`, relational `== != > <`, logical `&& || !`, assignment `+= -=`.
- `10 / 3 = 3` — int division truncates.
- `x++` use-then-add vs `++x` add-then-use.

Control Flow

- `if/else-if` for ranges & complex conditions.
- `switch` for exact-value matching — don't forget `break`.

Loops & Jumps

- **for**: known count. **while**: check first. **do-while**: runs ≥1 time (checks after).
- `break` exits loop, `continue` skips iteration, `return` exits method.

Methods & Constructors

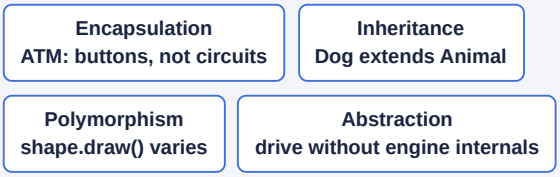
- Static method → call via `ClassName.m()`; instance method → via `obj.m()`.
- Constructor: no return type, same name as class, runs on `new`.

C1 OOP Introduction — the 4 Pillars

IN SHORT

- OOP models real things as **objects**: a **class** is the blueprint, an **object** is the instance.
- **Encapsulation** — bundle data+methods, hide internals (private fields, public getters/setters).
- **Inheritance** — a class acquires fields/methods from a parent (extends).
- **Polymorphism** — same method name, different behavior per object.
- **Abstraction** — hide implementation, expose only what's necessary.

MENTAL MODEL — THE 4 PILLARS, ONE ANALOGY EACH



Trick: mnemonic — "**E**very **I**nherited **P**illar **A**bstracts" (E-I-P-A).

Practice (s10): Airtribe Learner Manager — console CRUD app using parallel arrays, Scanner, switch, age-validation with if-else, average via a while-driven menu loop.

Class 2 — JVM Internals

What actually happens between `javac` and your program running.

C2 JDK, JRE & JVM

IN SHORT

- **JDK** = build & run (javac, javadoc, jar + JRE inside). Developers install this.
- **JRE** = run only (JVM + core libraries). No compiler.
- **JVM** = executes bytecode; gives platform independence, does GC.

Role	Needs
Developer	JDK
End user	JRE

MENTAL MODEL — NESTING DOLLS

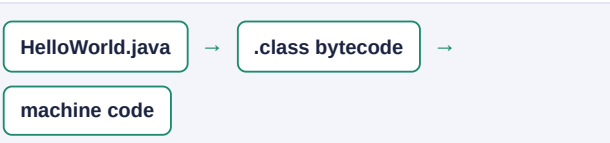
Trick: verbs in order — **D**evelop → **R**un → **E**xecute. "D-R-E, biggest to smallest."

C2 Compilation & Bytecode

IN SHORT

- Bytecode = platform-independent intermediate form, not human-readable.
- javac X.java → X.class; java X runs it; javap -c X.class disassembles it.
- Bytecode is **verified** before execution — a security layer.

MENTAL MODEL — THE PIPELINE



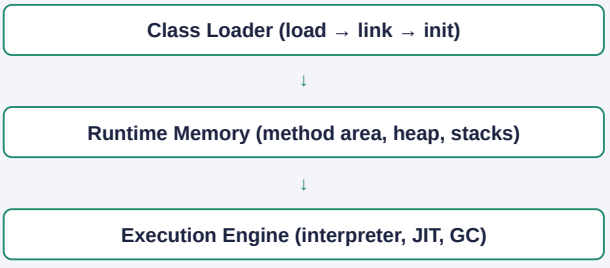
Trick: "compile once (javac), run anywhere (java) — inspect anytime (javap)."

C2 JVM Architecture

IN SHORT

- 4 blocks: **Class Loader** → **Runtime Memory** → **Execution Engine** → **Native Interface (JNI)**.
- Execution Engine = Interpreter + JIT Compiler + Garbage Collector.

MENTAL MODEL — THE ASSEMBLY LINE

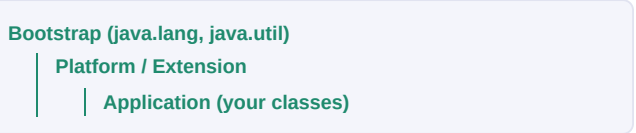


C2 Class Loading

IN SHORT

- 3 loaders: **Bootstrap** (core java.*) → **Platform/Extension** → **Application** (your classes).
- **Parent delegation**: a loader always asks its parent first; only loads it itself if the parent can't. Stops user code from shadowing core Java classes.
- 3 sub-steps overall: Loading → Linking → Initialization.

MENTAL MODEL — ASK YOUR PARENT FIRST



Trick: request flows UP to Bootstrap first, loading happens on the way back DOWN only if nobody above could do it.

C2 Linking Phase

IN SHORT

- **Verification** — is the .class file safe & well-formed (magic number, type safety)?
- **Preparation** — allocate static vars, set them to *default* values (0/false/null) — not your values yet.
- **Resolution** — convert symbolic references (class names as strings) into real memory addresses.

MENTAL MODEL — 3 CHECKPOINTS

Trick: "V-P-R." static int count=10; → is 0 after Prepare, 10 only after Initialization.

C2 Initialization

IN SHORT

- Final class-loading step: static vars get their *real* values, static blocks run — top to bottom, in source order.
- Superclass initializes before subclass. Static blocks run **once ever**, can't touch instance vars.
- Triggers: new, accessing a static field/method, Class.forName().

SNIPPET

```
static int x = 10;      // step 1
static { x = 20; }      // step 2 (block runs)
static int y = x + 5;   // step 3: y = 25
```

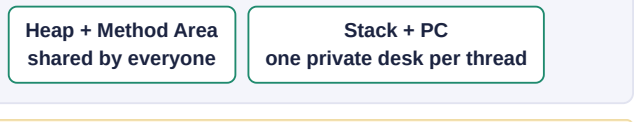
Trick: "top-to-bottom, once, parent-before-child."

C2 Runtime Memory Areas

IN SHORT

Area	Shared?	Stores	Fails with
Method Area	Yes	class metadata, statics	OutOfMemoryError
Heap	Yes	objects, instance vars	OutOfMemoryError
Java Stack	No (per thread)	call frames, locals	StackOverflowError
PC Register	No	current instruction	—
Native Stack	No	JNI method frames	StackOverflowError

MENTAL MODEL — SHARED OFFICE VS PERSONAL DESKS



Trick: infinite recursion blows the **Stack** (too many frames); too many live objects blows the **Heap**.

C2 JIT Compiler vs Interpreter

IN SHORT

- **Interpreter**: executes bytecode line-by-line. Fast startup, slow on repeats.
- **JIT**: compiles "hot" (frequently-run) bytecode to native code. Slower start, much faster after warm-up.
- JVM starts interpreting everything, then JIT-compiles the hot spots as it detects them.

MENTAL MODEL — WARMING UP

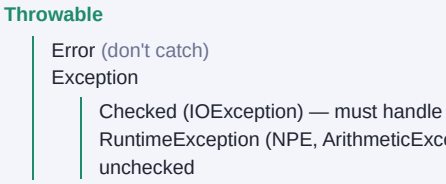
Trick: like a rehearsed speech — slow & careful the first time, fluent once memorized.

C2 Exception Handling

IN SHORT

- **Checked** (compile-time forced, e.g. IOException) vs **Unchecked/RuntimeException** (NullPointerException) vs **Error** (don't catch, e.g. OutOfMemoryError).
- try/catch/finally — finally always runs (cleanup).
- throws declares a checked exception a method might pass up.

MENTAL MODEL — THE HIERARCHY TREE



Trick: "Checked = compiler nags you. Unchecked = your bug. Error = not your job."

JVM Tools quick reference

Tool	Purpose
`javap -c`	view bytecode instructions
`java -verbose:class`	trace class loading
`jps -l`	list running Java processes
`jcmd <pid>`	JVM diagnostics (flags, heap, threads)
`jvisualvm`	GUI monitor: memory/CPU/threads/GC

Class 3 — Packages, Encapsulation & Access

Organizing code and controlling who can see what.

C3 Packages

IN SHORT

- A namespace container for classes — avoids name clashes (two different Teacher classes can coexist).
- package must be the first line in the file; lowercase by convention.
- Sub-packages are NOT auto-imported — import explicitly.

SNIPPET

```
school/math/Teacher.java     // package school.math
school/science/Teacher.java  // package
school.science
// same class name, different package — no clash!
```

Trick: packages are folders + a namespace label rolled into one.

C3 Encapsulation

IN SHORT

- Bundle data + methods into a class; hide internal state behind private fields.
- Access only via public getters/setters — setters can **validate**.

SNIPPET

```
public void setAge(int age) {
  if (age > 0 && age < 150) this.age = age;
  // invalid data silently rejected, not just stored
}
```

Trick: "no field is public unless you want anyone able to set it to garbage."

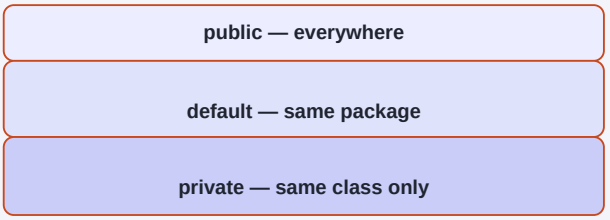
C3 Access Modifiers

VISIBILITY TABLE (PRIVATE → PUBLIC)

Visibility	private	default	public
Same class	YES	YES	YES
Same package	NO	YES	YES
Different package	NO	NO	YES

(protected = default + subclass access across packages — class 4.)

MENTAL MODEL — CONCENTRIC CIRCLES



Trick: start everything private, widen only when something outside genuinely needs it.

Practice (d): StudentReportCard — private fields, validated setters (0–100), getPercentage()/getGrade() derived methods, formatted printReportCard().

Class 4 — Inheritance & Polymorphism

Reusing and specializing behavior across classes.

Inheritance

- `extends` — child gets parent's fields/methods. Single, multilevel (A→B→C), hierarchical (1 parent, many children) all supported.
- **No multiple inheritance of classes** — avoids the Diamond Problem (which parent's method wins?).

Protected Access

- `protected` = default + visible to subclasses even in a **different package**.
- Use when children should touch parent fields directly, but randoms shouldn't.

this & super

- `this.x = x` disambiguates field vs param; `this(...)` chains constructors.
- `super(...)` calls parent constructor — must be the **first** line; can't use both `this()` and `super()` together.

Quick trick

- "Diamond problem" → why Java classes can't have 2 parents.
- "this = me, super = my parent." Constructors always call one or the other, first line, automatically if you don't.

C4 Polymorphism

IN SHORT

	Compile-time	Runtime
Achieved by	Overloading	Overriding
Resolved	compile time	runtime
Binding	early	late
Needs inheritance?	No	Yes

MENTAL MODEL

Overloading
same name, diff params
compiler decides

Overriding
same signature
JVM decides at runtime

Trick: "overLOADing = LOAD more versions (compile time). overRIDing = RIDE over the parent's version (runtime)."

Practice (e): Employee payroll — FullTimeEmployee/PartTimeEmployee extends Employee, override calculatePay(), overload displayInfo(), store mixed types in an Employee[] and watch polymorphism pick the right method.

Class 5 — Abstraction & Object Relationships

Hiding complexity, and how objects relate to and own each other.

C5 Abstraction

IN SHORT

- Hide the **how**, expose only the **what**. Two tools: abstract classes (0–100%) and interfaces (100%).
- Abstract class: mixes abstract+concrete methods, has fields/constructors, but **can't be instantiated**.

```
abstract class Shape {
  abstract double area();   // no body
  void describe() { ... }     // concrete
}
```

MENTAL MODEL — THE TV REMOTE

You press "Power" → **Remote (WHAT)** →

Circuitry (HOW, hidden)

Trick: ask "does the caller need the HOW, or just the WHAT?" If just WHAT → abstract it.

C5 Interfaces

IN SHORT

- Pure contract: abstract methods by default; since Java 8 also default/static methods.
- No constructors, no instance fields (only static final). A class can implement **multiple** interfaces — solves the diamond problem.

	Abstract class	Interface
Fields	instance vars OK	static+final only
Multi-inherit	No	Yes

SNIPPET

```
interface Drawable {
  void draw();   // abstract by default
  double area();
}
class Circle implements Drawable { ... }
```

Trick: interface = "contract for unrelated classes"; abstract class = "shared code for close relatives."

C5 Association, Aggregation & Composition

IN SHORT

	Association	Aggregation	Composition
Relation	"uses-a"	"has-a" weak	"has-a" strong
Child created	independent	outside, passed in	inside parent
Parent dies →	no effect	child survives	child dies too
Example	Driver↔Car	Library→Books	House→Rooms

```
// Aggregation: passed in from outside
library.addBook(new Book(...));
// Composition: created inside the parent
class House { rooms.add(new Room(...)); }
```

MENTAL MODEL — THE OWNERSHIP SPECTRUM

Trick — the lifecycle test: "can the part exist without the whole?" YES+created-outside → aggregation. NO+created-inside → composition. Neither owns → association.

Practice (f): Airtribe LMS — abstract Learner (abstraction) → JavaLearner/NodeJSLearner (inheritance); Cohort owns its Learners (composition) but only references an Instructor (aggregation); Course↔Cohort (association).

Class 6 — Design Principles & SOLID

Rules of thumb for code that stays maintainable as it grows.

C6 Design Principles — the big picture

IN SHORT

- Guidelines for code that is **maintainable, scalable, reusable, robust**.
- Coming up: **DRY**, **KISS**, **YAGNI**, the 5 **SOLID** principles, composition-over-inheritance.

MENTAL MODEL — COMPASS VS. MAP

Principles
= compass (direction)

Patterns
= map route (specific path)

Trick: principles tell you *what to aim for*; patterns show you *how to get there*.

C6 DRY, KISS & YAGNI

IN SHORT

Principle	Rule	Ask yourself
DRY	one source of truth	"Have I written this before?"
KISS	prefer the plain solution	"Can a junior read this in 10s?"
YAGNI	build only what's needed now	"Does anyone need this today?"

```
// KISS: guard clauses beat nesting
if (x == null) return;
if (x <= 0) return;
work();
```

MENTAL MODEL — THREE GUARDRAILS

DRY
no dup logic

KISS
no needless complexity

YAGNI
no speculative code

Trick: "Don't copy, Keep calm, You ain't need it yet." Violations: copy-pasted tax logic, 6-level if-else, an unused multiply().

SOLID — five principles for maintainable OOP

S Single Responsibility Principle

"A class should have only **one reason to change**." — Uncle Bob. An Employee class that stores employee data, calculates pay, saves to a database, **and** generates pay slips has three unrelated reasons to change: a tax-rule tweak, a database migration, or a pay-slip format change. Any one of those edits risks breaking the other two, even though they have nothing to do with each other.

BEFORE — ONE CLASS, THREE JOBS

```
class EmployeeBad {
  private String name;
  private double salary;

  public double calculatePay() {
    return salary - (salary * 0.1);
  }
  public void saveToDatabase() {
    // writes employee row to DB
  }
  public void generatePaySlip() {
    // prints a formatted pay slip
  }
}
// 3 reasons to change: tax rules, DB schema, slip format
```

AFTER — ONE JOB PER CLASS

```
class Employee {              // just data
  private String name;
  private double salary;
  // getters only
}
class PayCalculator {
  double calculatePay(Employee e) { … }
}
class EmployeeRepository {
  void save(Employee e) { … }
}
class PaySlipGenerator {
  void generate(Employee e) { … }
}
```

Why this fixes it: each class now changes for exactly one reason. A new tax rule only touches PayCalculator; switching from MySQL to Mongo only touches EmployeeRepository — neither can accidentally break the other.

MENTAL MODEL — THE RESTAURANT

Chef cooks | Waiter serves | Cashier bills

Trick: describe the class in one sentence. If it needs "AND", split it. "Manages employees **AND** sends emails" = violation.

O Open/Closed Principle

Classes should be **open for extension** but **closed for modification**. PaymentProcessorBad.processPayment() branches on a string — UPI, CreditCard, DebitCard. Every new payment method means reopening that method and risking the branches that already work. OCP says: add new behavior by *writing new code*, never by editing code that's already correct.

BEFORE — IF-ELSE ANTI-PATTERN

```
class PaymentProcessorBad {
  void processPayment(String type, double amt) {
    if (type.equals("UPI")) { … }
    else if (type.equals("CreditCard")) { … }
    else if (type.equals("DebitCard")) { … }
    // NetBanking? must MODIFY this method,
    // risking UPI/Card logic that already works
  }
}
```

AFTER — INTERFACE-BASED

```
interface PaymentMethod { void pay(double amt); }

class UpiPayment implements PaymentMethod { … }
class CreditCardPayment implements PaymentMethod { …
}
// added later — NOTHING above was touched
class NetBankingPayment implements PaymentMethod { …
}

class PaymentProcessor {
  void processPayment(PaymentMethod m, double amt) {
    m.pay(amt);  // works for ANY payment method
  }
}
```

Why this fixes it: PaymentProcessor never changes again. Adding NetBankingPayment took one new class; UpiPayment and CreditCardPayment were never reopened or re-tested.

MENTAL MODEL — THE USB PORT

USB port (never rewired) ← any device plugs in

Trick: "New variant? New class. Done." If you're reopening a working class to add a branch, you're breaking OCP.

L Liskov Substitution Principle

Any subclass must be safely usable wherever the parent is expected — no surprises. BirdBad.fly() is a promise every bird can fly. OstrichBad extends BirdBad but can't actually fly, so any code that calls bird.fly() on what it believes is a plain Bird can crash for an Ostrich. The hierarchy itself is the bug — not the calling code.

BEFORE — BROKEN PROMISE

```
class BirdBad {
  void fly() { /* flying logic */ }
}
class OstrichBad extends BirdBad {
  void fly() {
    throw new UnsupportedOperationException(
      "Ostriches can't fly!");  // VIOLATION
  }
}
void makeBirdFly(BirdBad bird) {
  bird.fly();  // crashes if bird is an Ostrich!
}
```

AFTER — A CONTRACT EVERY BIRD CAN KEEP

```
abstract class Bird {
  abstract void move();  // ALL birds can move
}
class Sparrow extends Bird {
  void move() { fly(); }
}
class Ostrich extends Bird {
  void move() { walk(); }  // valid, no exception!
}
void makeBirdMove(Bird bird) {
  bird.move();  // safe for ANY bird
}
```

Why this fixes it: move() is a promise every bird can actually keep. No subclass throws, no subclass needs an empty override — because the contract was redesigned to fit every child, not just the "typical" one.

MENTAL MODEL — THE SWAP TEST

Expects Parent → Swap in Child → Still works?

Trick: a "driver" who says "can't drive manual" broke the contract. Red flags: a child that throws, does nothing, or adds surprise preconditions.

I Interface Segregation Principle

Clients shouldn't be forced to depend on methods they don't use. WorkerBad bundles work(), eat(), and sleep() into one interface. A HumanWorker can do all three, but a RobotWorker is forced to implement eat() and sleep() too — even though a robot does neither — and ends up throwing exceptions just to satisfy a contract it never wanted.

BEFORE — ONE FAT INTERFACE

```
interface WorkerBad {
  void work();
  void eat();
  void sleep();
}
class RobotWorkerBad implements WorkerBad {
  public void work() { … }
  public void eat() {
    throw new UnsupportedOperationException(
      "Robots don't eat!");   // forced!
  }
  public void sleep() {
    throw new UnsupportedOperationException(
      "Robots don't sleep!");  // forced!
  }
}
```

AFTER — SMALL, FOCUSED INTERFACES

```
interface Workable  { void work();  }
interface Eatable   { void eat();   }
interface Sleepable { void sleep(); }

class HumanWorker implements
    Workable, Eatable, Sleepable { … }

class RobotWorker implements Workable {
  public void work() { … }
  // no eat(), no sleep() — nothing to fake!
}
```

Why this fixes it: RobotWorker simply doesn't implement Eatable or Sleepable — there's no method to fake, no exception to throw. The compiler enforces honesty about what a class can actually do.

MENTAL MODEL — THE MENU

50-item menu (fat) → Veg / Non-veg menus

Trick: spot the smell — a method that's just throw new UnsupportedOperationException() or a comment like "doesn't apply here."

D Dependency Inversion Principle

High-level modules shouldn't depend on low-level modules directly — both should depend on an abstraction. SwitchBad creates its own LightBulb inside its constructor (new LightBulb()), hardcoding it to one specific device. To control a Fan instead, you'd have to rewrite Switch itself. DIP fixes this with **dependency injection**: the concrete device is handed to Switch from outside.

BEFORE — TIGHTLY COUPLED

```
class LightBulb {
  void turnOn() { … }
}
class SwitchBad {
  private LightBulb bulb;
  SwitchBad() {
    this.bulb = new LightBulb();  // hardcoded!
  }
  void operate() { bulb.turnOn(); }
  // want a Fan instead? must rewrite this class!
}
```

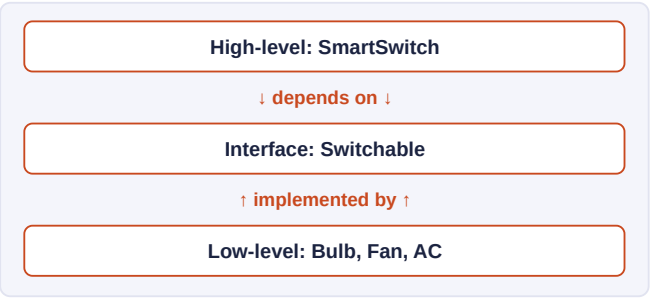
AFTER — DEPENDS ON AN INTERFACE

```
interface Switchable {
  void turnOn(); void turnOff();
}
class SmartBulb implements Switchable { … }
class Fan implements Switchable { … }
class AC implements Switchable { … }

class SmartSwitch {
  private Switchable device;
  SmartSwitch(Switchable device) {
    this.device = device;  // injected!
  }
  void operate() { device.turnOn(); }
}
```

Why this fixes it: the same SmartSwitch now controls a bulb, a fan, or an AC — just pass a different Switchable in: new SmartSwitch(new Fan()). SmartSwitch never mentions SmartBulb, Fan, or AC by name, only the interface.

MENTAL MODEL — INVERSION



Trick: USB-C charger — your phone depends on the plug shape (the standard), not the brand of charger.

Composition over Inheritance

Prefer "has-a" (compose behavior from parts) over "is-a" (inherit a rigid hierarchy). DuckBad hardwires swim(), quack(), and fly() into one base class. RubberDuckBad extends DuckBad but can't really fly, so its fly() override does nothing — an empty override, the same LSP smell from before. Every new duck variant means more overrides and a more fragile hierarchy.

BEFORE — INHERITANCE, RIGID

```
class DuckBad {
  void swim() { … }
  void quack() { … }
  void fly() { … }
}
class RubberDuckBad extends DuckBad {
  void quack() { /* "Squeak!" instead */ }
  void fly() { /* empty — can't fly! */ }
}
```

AFTER — COMPOSITION, FLEXIBLE

```
interface SwimBehavior  { void swim(); }
interface SoundBehavior { void makeSound(); }
interface FlyBehavior   { void fly(); }
// CanFly/CannotFly, Quack/Squeak/Silent implement
these

class Duck {
  SwimBehavior swim; SoundBehavior sound; FlyBehavior
fly;
  Duck(SwimBehavior s, SoundBehavior snd, FlyBehavior
f) {
    this.swim=s; this.sound=snd; this.fly=f;  // HAS-
A
  }
}
new Duck(new CanSwim(), new Squeak(), new
CannotFly());
```

Why this fixes it: no empty overrides, no LSP violations. A rubber duck is built by picking the right behaviors *at construction time*, not by inheriting everything and disabling what it doesn't want.

MENTAL MODEL — THE SWAPPABLE ENGINE

Car IS NOT an engine → Car HAS an engine (swap it!)

Use inheritance	Use composition
true IS-A, shallow hierarchy	behavior changes at runtime

Class 8 — Applying OOP Relationships

Taking association/aggregation/composition & composition-over-inheritance from theory to a real system.

C8 Deciding Association vs. Aggregation vs. Composition

IN SHORT — FROM THE FOOD DELIVERY SYSTEM BUILD

- **Restaurant ↔ MenuItem → Aggregation.** If the restaurant shuts down, "Butter Chicken" can still exist on another menu. Items are created outside and addMenuItem()-ed in.
- **Order ↔ OrderItem → Composition.** "2x Butter Chicken in Order #1001" is meaningless outside that order. The Order creates its OrderItems internally (constructor is package-private).
- **Customer ↔ Order** and **DeliveryAgent ↔ Order → Association.** Independent lifecycles, no ownership either way.

MENTAL MODEL — THE LITMUS QUESTION

"If the parent disappears tomorrow, does the child still make sense?"

YES, and it was passed in → Aggregation

NO, it was created inside → Composition

Neither owns the other → Association

Trick: two enums (Category, OrderStatus) instead of strings — compiler catches typos, autocomplete works.

Also in class 8: a_composition_over_inheritance re-runs the Class 6 principle against a concrete inheritance-gone-rigid problem; b_association_aggregation_composition re-demos all three relationships standalone before the practice problem combines them.

Practice (c): Online Food Delivery System — Restaurant, MenuItem, Customer, Order/OrderItem, DeliveryAgent, a PaymentMethod abstraction (CreditCardPayment/UPIPayment), a Trackable interface, and Category/OrderStatus enums — the relationships above, all in one system.

Class 9 — Creational Design Patterns

Proven, named solutions to "how should I create this object?"

C9 Intro to Design Patterns

IN SHORT

- A pattern is a proven, reusable **idea** (not code) with a **name** — shared vocabulary. From the "Gang of Four" (1994).
- **Principles** = grammar of good code. **Patterns** = sentences built with that grammar.

Family	Concerned with	Examples
Creational	how objects are created	Singleton, Builder, Factory
Structural	how objects compose	Decorator, Adapter
Behavioral	how objects interact	Observer, Strategy

MENTAL MODEL — THE DECISION MAP

Only ONE in the whole program? → Singleton

Lots of optional knobs? → Builder

Choice depends on runtime input? → Factory

Need a whole MATCHED FAMILY? → Abstract Factory

C9 Singleton

Ensure a class has exactly **one instance** for the whole program, with one global access point. If the constructor is public, nothing stops five different classes from each creating their own Logger or ConfigManager — defeating the point of having a single shared one. The fix: a **private** constructor (nobody can call new), a **private static** field to hold the one instance, and a **public static** getInstance() that everyone calls instead.

LAZY — NOT THREAD-SAFE

```
class LazySingleton {
  private static LazySingleton instance;
  private LazySingleton() {}

  public static LazySingleton getInstance() {
    if (instance == null)
      instance = new LazySingleton();
    // two threads can BOTH pass this check
    // before either finishes — TWO instances!
    return instance;
  }
}
```

DOUBLE-CHECKED LOCKING — LAZY + SAFE

```
class ThreadSafeSingleton {
  private static ThreadSafeSingleton instance;
  private ThreadSafeSingleton() {}

  public static ThreadSafeSingleton getInstance() {
    if (instance == null) {           // 1st check,
no lock
      synchronized (ThreadSafeSingleton.class) {
        if (instance == null) {       // 2nd check,
in lock
          instance = new ThreadSafeSingleton();
        }
      }
    }
    return instance;
  }
}
```

Why this fixes it: the first null check avoids grabbing a lock on every single call (fast path, once the instance exists). The second null check happens *inside* the lock, so if two threads both got past the first check simultaneously, only one of them actually creates the instance — the other finds it already set.

MENTAL MODEL — THE SCHOOL PRINCIPAL

Many teachers, many students →

exactly ONE Principal

Trick: "One instance, private constructor, static field, static getter." Real uses: Logger, DB connection pool, app config.

C9 Builder

HttpRequest has 2 required fields (method, url) and 4 optional ones (body, timeout, followRedirects, authToken). A constructor covering every combination either explodes into overloads, or turns into new HttpRequest("POST", url, null, 30, true, null) — unreadable, and easy to swap two arguments by mistake. Builder fixes this: required fields go in the Builder's constructor, optional fields get fluent setters with sensible defaults, and build() validates before handing back an **immutable** object.

PRODUCT + BUILDER

```
class HttpRequest {
  private final String method, url;   // required
  private String body; private int timeoutSeconds;
  private HttpRequest(Builder b) {   // private!
    this.method=b.method; this.url=b.url;
    this.body=b.body; …
  }

  static class Builder {
    private final String method, url;
    private String body = null;          // defaults
    private int timeoutSeconds = 30;

    Builder(String method, String url) {
      this.method=method; this.url=url;
    }
    Builder body(String b){ this.body=b; return this;
}
    Builder timeout(int s){ timeoutSeconds=s; return
this; }

    HttpRequest build() {
      if (method == null) throw new
IllegalStateException("method required");
      return new HttpRequest(this);
    }
  }
}
```

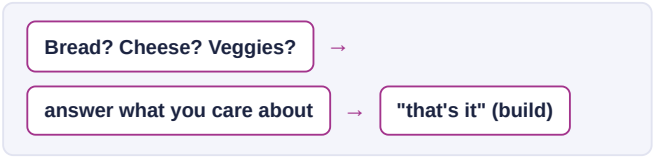
USAGE

```
HttpRequest post =
  new HttpRequest.Builder("POST", ".../users")
    .body("{\"name\":\"Alice\"}")
    .timeout(10)
    .authToken("Bearer abc123")
    .build();

// vs. the telescoping-constructor problem:
// new HttpRequest("POST", url, body,
//     10, true, "Bearer abc123")
// — which boolean is which? impossible to tell.
```

Why this fixes it: every setter returns this, so calls chain fluently and each one is *named* — .timeout(10) can't be confused with .followRedirects(10). build() is the one place that validates, and the finished HttpRequest has no public setters, so it can never be mutated after creation.

MENTAL MODEL — THE SUBWAY SANDWICH



Trick: real uses — StringBuilder, OkHttpClient.Builder, AlertDialog.Builder.

C9 Factory Method

Every place that needs a notifier writes new EmailNotifier() or new SMSNotifier() directly. Add a PushNotifier and you have to hunt down and edit every one of those call sites — the same "new everywhere" coupling problem OCP warned about. A factory centralizes the decision: callers ask for a Notifier by name, and only the factory knows the concrete classes.

A) STATIC FACTORY METHOD

```
interface Notifier { void send(String m); }
class EmailNotifier implements Notifier { … }
class SMSNotifier implements Notifier { … }

class NotifierFactory {
  static Notifier create(String channel) {
    return switch (channel) {
      case "email" -> new EmailNotifier();
      case "sms"   -> new SMSNotifier();
      default      -> throw new
IllegalArgumentException();
    };
  }
}
// caller: Notifier n =
NotifierFactory.create("email");
```

B) GOF FACTORY METHOD

```
abstract class NotificationService {
  // the factory method — subclass decides the type
  protected abstract Notifier createNotifier();

  // template: uses it without knowing the type
  public void notifyUser(String event) {
    Notifier n = createNotifier();
    n.send(event);
  }
}
class EmailNotificationService
    extends NotificationService {
  Notifier createNotifier() { return new
EmailNotifier(); }
}
```

Why this fixes it: in **A**, adding PushNotifier means editing one switch in one file — every caller of NotifierFactory.create() is untouched. In **B**, adding a channel means adding a new NotificationService subclass — notifyUser() never changes. Either way, callers only ever see the Notifier interface.

MENTAL MODEL — THE BARISTA

Trick: "Factory hides new. Caller names a product; factory decides the class."

C9 Abstract Factory

A UI has a Button, a Checkbox, and a TextField. If you create a MacButton but a WinCheckbox by mistake, the UI looks broken — components from different platforms were never meant to mix. Abstract Factory groups creation into one factory *per family*, so picking a factory picks an entire consistent set at once. Think "factory of factories": Factory Method makes **one** product, Abstract Factory makes a **matched suite**.

PRODUCTS + FACTORIES

```
interface Button   { void render(); }
interface Checkbox { void render(); }
class MacButton implements Button { … }
class MacCheckbox implements Checkbox { … }
class WinButton implements Button { … }
class WinCheckbox implements Checkbox { … }

interface UIFactory {
  Button createButton();
  Checkbox createCheckbox();
}
class MacUIFactory implements UIFactory {
  Button createButton() { return new MacButton(); }
  Checkbox createCheckbox() { return new
MacCheckbox(); }
}
// WindowsUIFactory mirrors this, returning Win*
instead
```

CLIENT — NEVER SEES MAC/WIN DIRECTLY

```
class LoginForm {
  private final Button button;
  private final Checkbox checkbox;

  LoginForm(UIFactory factory) {   // injected
    this.button = factory.createButton();
    this.checkbox = factory.createCheckbox();
  }
}

// one swap changes the WHOLE ui, consistently:
new LoginForm(new MacUIFactory());
new LoginForm(new WindowsUIFactory());
```

Why this fixes it: LoginForm never imports MacButton or WinButton — only UIFactory, Button, Checkbox. Swapping factories changes the entire UI consistently, and adding a LinuxUIFactory later needs zero changes to LoginForm.

MENTAL MODEL — PHONE THEME

Dark mode ON → **button, dialog, icons ALL dark**

Trick: Factory Method makes **one** product; Abstract Factory makes a **matched set**, picked once.

Practice (e / warmup): Cloud Storage SDK (Singleton ConfigManager, Builder StorageRequest, Factory StorageClientFactory, Abstract Factory CloudFactory bundling client+logger per provider) and the "Slice of Heaven" pizza warmup (singleton shop settings, builder custom pizza, factory presets, abstract factory themed combos).

Structural & Behavioral Design Patterns

Five more GoF patterns, not tied to a specific class session in the source repo.

Observer

Define a one-to-many dependency so that when one object changes, everyone depending on it is notified automatically. Without it, a Dashboard, an AlertService, and a Logger would each have to keep **polling** the stock price ("has it changed yet?") — wasteful, and it couples every watcher to Stock's internals. Observer flips it: Stock keeps a list of Observers and pushes updates to all of them the moment its price changes.

SUBJECT + OBSERVERS

```
interface Observer {
  void update(String name, double price);
}
class Stock {
  private final List<Observer> observers = new
ArrayList<>();
  private double price;

  void addObserver(Observer o) { observers.add(o); }

  void setPrice(double newPrice) {
    this.price = newPrice;
    for (Observer o : observers)
      o.update(name, price);      // push to everyone
  }
}
```

CONCRETE OBSERVERS + USAGE

```
class DashboardDisplay implements Observer {
  public void update(String n, double p) { … }
}
class PriceAlertService implements Observer {
  public void update(String n, double p) {
    if (p < threshold) alert();
  }
}

Stock apple = new Stock("AAPL", 150.0);
apple.addObserver(new DashboardDisplay());
apple.addObserver(new PriceAlertService(140.0));
apple.setPrice(135.0);  // BOTH notified
automatically
```

Why this fixes it: Stock doesn't know or care what a DashboardDisplay or PriceAlertService actually does with the update — it just calls update() on everyone registered. Observers can be added or removed at runtime without touching Stock at all.

MENTAL MODEL

Trick: "Don't call us, we'll call you." Real use: GUI listeners, Kafka pub/sub. Pitfall: forgetting to unregister → memory leak.

Strategy

Define a family of interchangeable algorithms and make them swappable at runtime. Without it, ShoppingCart.checkout() would branch on payment type — if (type.equals("credit")) … else if (type.equals("paypal")) … — the same OCP problem as a growing if-else chain. Strategy pulls each algorithm into its own class implementing a common interface; the Context just delegates to whichever one it's holding.

STRATEGY INTERFACE + IMPLEMENTATIONS

```
interface PaymentStrategy {
  void pay(double amount);
}
class CreditCardPayment implements PaymentStrategy {
  public void pay(double amt) { … }
}
class PayPalPayment implements PaymentStrategy {
  public void pay(double amt) { … }
}
```

CONTEXT + USAGE

```
class ShoppingCart {
  private PaymentStrategy strategy;

  void setPaymentStrategy(PaymentStrategy s) {
this.strategy = s; }
  void checkout(double total) {
    strategy.pay(total);   // delegates, doesn't know
HOW
  }
}

cart.setPaymentStrategy(new
CreditCardPayment("4111..."));
cart.checkout(99.99);
cart.setPaymentStrategy(new
PayPalPayment("a@x.com"));  // swapped!
cart.checkout(49.50);
```

Why this fixes it: checkout() never changes when a new payment method is added — only a new PaymentStrategy class is written. The strategy can even be swapped mid-program, between two checkouts on the same cart.

MENTAL MODEL

Trick: "Same job, different ways to do it, pick one at runtime." Real use: Comparator<T>.

Chain of Responsibility

Pass a request along a chain of handlers; each handler decides to either process it or forward it. A support ticket needs routing to whichever tier can handle it — FAQ bot for simple issues, junior agent for medium, senior engineer for hard ones. A single dispatcher with nested severity checks would need editing every time a new tier is added. Chain of Responsibility links handlers together instead, so the sender doesn't need to know which one will actually process it.

ABSTRACT HANDLER

```
abstract class SupportHandler {
  private SupportHandler next;

  SupportHandler setNext(SupportHandler n) {
    this.next = n; return n;   // fluent chaining
  }
  void handle(SupportTicket t) {
    if (canHandle(t)) process(t);
    else if (next != null) next.handle(t);
    else System.out.println("No handler for: " + t);
  }
  abstract boolean canHandle(SupportTicket t);
  abstract void process(SupportTicket t);
}
```

CONCRETE HANDLERS + USAGE

```
class FaqBot extends SupportHandler {
  boolean canHandle(SupportTicket t) { return
t.getSeverity()==1; }
  void process(SupportTicket t) { /* auto-resolve */
}
}
// JuniorAgent → severity 2, SeniorEngineer →
severity 3

SupportHandler faqBot = new FaqBot();
faqBot.setNext(new JuniorAgent())
      .setNext(new SeniorEngineer());

faqBot.handle(new SupportTicket("DB corrupted!", 3));
// walks the chain until SeniorEngineer handles it
```

Why this fixes it: the client always calls handle() on the first handler and doesn't know or care which one actually processes it. Adding a Level4 escalation tier means one more class and one more setNext() call — no existing handler changes.

MENTAL MODEL

Trick: "Pass the buck until someone handles it." Real use: servlet filters. Pitfall: no fallback → silently dropped request.

Decorator

Attach responsibilities to an object dynamically, without modifying its class. A coffee shop wants coffee with any combination of milk, sugar, and whipped cream. Subclassing every combination (CoffeeWithMilk, CoffeeWithMilkAndSugar…) explodes to 2^N classes for N toppings. Decorator wraps a Coffee inside another object implementing the *same interface*, which adds its own cost/description and delegates the rest — toppings stack like layers instead of multiplying as subclasses.

COMPONENT + DECORATORS

```
interface Coffee {
  String getDescription(); double getCost();
}
class SimpleCoffee implements Coffee {
  String getDescription() { return "Simple coffee"; }
  double getCost() { return 2.00; }
}
abstract class CoffeeDecorator implements Coffee {
  protected final Coffee wrapped;
  CoffeeDecorator(Coffee c) { this.wrapped = c; }
}
class MilkDecorator extends CoffeeDecorator {
  MilkDecorator(Coffee c) { super(c); }
  String getDescription() { return
wrapped.getDescription()+" + milk"; }
  double getCost() { return wrapped.getCost() + 0.50;
}
}
```

STACKING THEM + USAGE

```
// SugarDecorator mirrors MilkDecorator, +0.25

Coffee order = new SugarDecorator(
                    new MilkDecorator(
                      new SimpleCoffee()));

order.getCost();
// = 2.00 + 0.50 + 0.25 = 2.75
order.getDescription();
// = "Simple coffee + milk + sugar"

// each decorator only knows the Coffee interface —
// you can wrap a decorator in another decorator
```

Why this fixes it: 3 decorator classes (Milk, Sugar, WhipCream) give you every possible combination, not 2^3 hardcoded subclasses. Adding a 4th topping is one new decorator class, not a doubling of existing ones.

MENTAL MODEL

Trick: "wrap it like a gift, each layer adds something." Real use: BufferedReader(new InputStreamReader(...)). Watch: order can matter, instanceof breaks.

State

Let an object alter its behavior when its internal state changes — the object appears to change its class. A Document moves through Draft → Review → Published, and what submit()/approve() do depends entirely on the current stage. Writing that as if (state==DRAFT) … else if (state==REVIEW) … inside Document means editing that block for every new stage. State pulls each stage into its own class — behavior changes simply by swapping which state object is held.

STATE INTERFACE + CONCRETE STATES

```
interface DocumentState {
  void submit(Document doc);
  void approve(Document doc);
}
class DraftState implements DocumentState {
  public void submit(Document doc) {
    System.out.println("Submitted for review.");
    doc.setState(new ReviewState());  // transition!
  }
  public void approve(Document doc) {
    System.out.println("Can't approve — not
submitted.");
  }
}
```

CONTEXT + USAGE

```
class Document {
  private DocumentState state = new DraftState();

  void setState(DocumentState s) { state = s; }
  void submit()  { state.submit(this); }   // just
delegates
  void approve() { state.approve(this); }
}

Document doc = new Document();
doc.submit();   // Draft → Review
doc.approve();  // Review → Published
```

Why this fixes it: Document itself has zero if-else — it just forwards every call to whatever state it currently holds. Each state class only needs to know its own transitions; adding a stage (e.g. ArchivedState) means adding one class.

STRATEGY VS. STATE

	Strategy	State
Who switches?	client, explicitly	state switches itself
Purpose	swap algorithm	model a lifecycle

Trick: "Context is the machine, each state is a mode." Real use: order lifecycle, TCP connection states.

Practice Questions — Design Patterns

2–3 questions per pattern — mix of "spot the problem," refactor, and design-from-scratch.

Singleton

- Two threads might create your `Logger` simultaneously on first use. Which variant fixes this, and why does plain lazy init fail?
- A teammate gives `Singleton` a `public` constructor "just in case." What breaks, and how do you still allow tests to inject a mock?
- Design a `ConfigManager` singleton (region, provider, debug flag) using double-checked locking — write the field, constructor, and `getInstance()`.

Builder

- Rewrite `new Pizza("large","thin",true,List.of("olives"),2,false)` using a Builder.
- Why should the built `Product` have no public setters? What breaks if it does?
- Design `StorageRequest.Builder`: required `bucket` / `key` in the constructor, optional `encrypted` / `contentType` chained, validation in `build()`.

Factory Method

- Adding `SlackNotifier` to `EmailNotifier` / `SMSNotifier`: which files change with Static Factory vs. Factory Method (abstract creator)? Compare.
- Refactor an `if (type.equals("email")) ... else if ...` block repeated across five call sites into a factory.
- Write `PaymentHandlerFactory.create(String method)` for `"upi"/"card"/"paypal"`; explain why callers should only ever see `PaymentHandler`.

Abstract Factory

- Explain, with an example, why mixing `MacButton` with `WinCheckbox` is exactly the bug this pattern prevents.
- Design a `CloudFactory` for AWS/GCP/Azure, each producing a matched `StorageClient` + `Logger`. Sketch the interface and one concrete factory.
- When would plain Factory Method be enough, and Abstract Factory overkill? Give a one-sentence rule.

Observer

- A `Stock` notifies 200 observers per tick, but closed UIs never unregister. Name the bug and fix it.
- Compare polling ("check every second") to Observer — which is more efficient, and why?
- Design a `WeatherStation` notifying `PhoneDisplay` and `WebDashboard` on temperature change — sketch the interfaces and the notify loop.

Strategy

- `ShoppingCart.checkout()` has a growing `if (type.equals("credit"))...` chain. Refactor it with Strategy.
- What's the key difference between Strategy and Factory? (One creates objects; the other swaps …?)
- Design a `SortingContext` that switches between `QuickSortStrategy` and `MergeSortStrategy` at runtime.

Chain of Responsibility

- Design a support-ticket chain: `Level1Handler` → `Level2Handler` → `Level3Handler`. Write `handle()`.
- What happens if a ticket reaches the end of the chain unhandled? How do you guard against that silently?
- vs. one long if-else for the same problem — what does the chain buy you when a new handler is added?

Decorator

- From `SimpleCoffee`, write the decorator stack for "milk + sugar + whipped cream" and its `getCost()`.
- Why does `instanceof SimpleCoffee` break after wrapping it? What does that imply about relying on concrete types?
- Name a JDK class that uses Decorator and identify which role (Component/ConcreteComponent/Decorator) it plays.

State

- Model a `TrafficLight` (RED→GREEN→YELLOW→RED) — sketch the state interface and one transition.
- A junior dev writes `if (status.equals("RED")) ... else if ...` inside `TrafficLight` itself. Which principle does this violate, and how does State fix it?
- In one sentence, how does State differ from Strategy, using the Draft→Review→Published example?

Appendix — Practice Problems Index

Class	Problem	Concepts practiced
1 (s10)	Airtribe Learner Manager (console)	arrays, Scanner, switch, if-else validation, while-loop menu
2 (s11)	Learner Manager, revisited	same LMS exercise, reinforcing fundamentals after JVM internals
3 (d)	Student Report Card System	encapsulation, validated setters, derived getPercentage()/getGrade()
4 (e)	Employee Payroll System	inheritance, super() chaining, method overriding & overloading, polymorphic array
5 (f)	Airtribe LMS (OOP)	abstraction, interfaces, encapsulation, inheritance, composition, aggregation, association — all together
8 (c)	Online Food Delivery System	choosing association vs. aggregation vs. composition deliberately; enums; Trackable interface; payment abstraction
9 (e)	Cloud Storage SDK	Singleton config, Builder request, Factory client, Abstract Factory client+logger family
9 (warmup)	"Slice of Heaven" Pizza App	all 4 creational patterns applied to one small, concrete domain